

Advanced RAG: Improving Retrieval Quality in Spring AI

Coder Army

1. Why Basic RAG Is Not Always Enough

A customer asks an e-commerce chatbot: **"What is your return policy?"** A general-purpose LLM may not know the company's internal policies. In a basic Retrieval-Augmented Generation (RAG) application, we search the company documents, retrieve relevant information and pass it to the LLM along with the question.

The basic pipeline is:

```
Customer question
↓
Generate query embedding
↓
Vector similarity search
↓
Retrieve Top-K chunks
↓
Pass question + retrieved context to LLM
↓
Generate answer
```

This approach works well for simple questions, but it depends on a critical assumption: **the retrieved chunks contain all the information needed to answer correctly.**

Example: A return-policy question

Consider these fictional company policies:

Document	Policy
General Return Policy	Eligible products may be returned within 30 days of delivery.
Electronics Return Policy	Electronic products may be returned within 7 days of delivery, subject to inspection.

Document	Policy
Damaged Product Policy	Customers must report a damaged product within 48 hours of delivery.

A customer asks: **"Can I return my damaged headphones after 10 days?"**

The answer depends on more than the general return window. Headphones are electronics, the product is damaged, and 10 days have passed. If the vector search retrieves only the general policy, the LLM sees the 30-day rule and may say that a return is allowed. That answer is understandable from the context supplied to it, but it misses two important policy conditions.

A good LLM cannot reliably use a policy that never reaches its context.

Two different RAG failures

Failure	What went wrong	Example	Where to investigate
Retrieval failure	The required information was not retrieved.	The system finds the general policy but misses the electronics policy.	Chunking, search strategy, retrieval parameters and filters.
Generation failure	The information was retrieved, but the model used it incorrectly or incompletely.	The model receives both policies but follows the general rule rather than the electronics-specific condition.	Context arrangement, instructions and response generation.

These problems need different fixes. A stronger system prompt cannot recover a missing document. Similarly, retrieving more documents does not guarantee that the model will interpret them properly.

What "Advanced RAG" means

Advanced RAG is not a replacement for RAG. It improves individual stages of the existing pipeline.

Problem	Improvement
Important information gets split apart or loses context	Better chunking

Problem	Improvement
Irrelevant documents appear in search results	Retrieval tuning and filtering
Exact identifiers or keywords are missed	Hybrid search
Useful documents are present but ranked too low	Reranking
Retrieved context is repetitive or contains competing information	Context engineering

Practical: Inspect retrieval before inspecting the answer

In the current Spring AI application, the initial retrieval call looks like this:

```
List<Document> relevantChunks = vectorStore.similaritySearch(
    SearchRequest.builder()
        .query(question)
        .topK(4)
        .build()
);
```

Temporarily print the retrieved text:

```
for (int i = 0; i < relevantChunks.size(); i++) {
    Document document = relevantChunks.get(i);

    System.out.println("DOCUMENT " + (i + 1));
    System.out.println(document.getText());
    System.out.println("-----");
}
```

Ask a few questions whose answers are available in the knowledge base. Check what Pinecone returned **before** looking at the LLM's final answer. This creates a baseline against which to compare later improvements.

2. Advanced Chunking Strategies

2.1 Why chunk documents?

Suppose the company has a 100-page document covering returns, refunds, shipping, warranties, exchanges and cancellation rules. Creating one embedding for the entire document—assuming it fits the embedding model's input limit—would force a single vector to represent many unrelated subjects.

For a specific question such as **“What is the return period for electronics?”**, that representation may be too broad to retrieve precisely. Chunking divides the document into smaller, searchable units. The important design question is **where to split without losing meaning**.

2.2 Fixed-size chunking

A fixed-size strategy splits text after a chosen number of characters or tokens. It is simple and produces predictable chunk lengths, but a token boundary need not coincide with a meaningful sentence or policy rule.

Original policy:

Electronics Return Policy

Electronic products can be returned within 7 days of delivery. Products must be in their original packaging. Items showing signs of physical damage caused by the customer are not eligible for return. Customers must provide the original purchase invoice.

An illustrative split might separate it into:

Chunk 1

Electronic products can be returned within 7 days of delivery. Products must be in their original packaging.

Chunk 2

Items showing signs of physical damage caused by the customer are not eligible for return. Customers must provide the original purchase invoice.

A question about a physically damaged electronic product may require **both** chunks: one states the return window, while the other states an exception. Retrieving only one can produce an incomplete answer.

Chunk overlap

Overlap repeats a small amount of text between neighboring chunks to retain information close to a split boundary. It is useful when a sentence or condition spans two chunks, but excessive overlap creates more embeddings, increases storage and can return repetitive results.

As an **experiment**, compare chunks of roughly **400–600 tokens with 10–20% overlap** against non-overlapping chunks. These values are starting points, not universal settings.

2.3 Structural or document-aware chunking

Many documents already have useful boundaries: headings, subheadings, paragraphs, sections and tables. Structural chunking respects these boundaries instead of splitting blindly.

Retain the heading when creating a chunk:

```
Return Policy → Electronics
```

```
Electronic products can be returned within 7 days.
```

The heading matters. Without it, “within seven days” might refer to a return, cancellation, warranty registration or refund process.

For a small collection of policy sections, each self-contained section can be added as an individual Spring AI `Document`:

```
List<Document> documents = List.of(
    new Document(
        """
        Return Policy → Electronics

        Electronic products can be returned
        within 7 days of delivery.
        """,
        Map.of(
            "source", "return-policy",
```

```

        "category", "electronics",
        "policyType", "returns"
    )
),
new Document(
    """
    Return Policy → Clothing

    Clothing products can be returned
    within 30 days of delivery.
    """,
    Map.of(
        "source", "return-policy",
        "category", "clothing",
        "policyType", "returns"
    )
)
);

vectorStore.add(documents);

```

For long sections, first separate content using headings, then apply a token splitter to any unusually large section. **Preserve meaningful boundaries first; enforce size limits second.**

2.4 Semantic chunking

Structural chunking is helpful when documents have clear headings. Semantic chunking is useful to evaluate when a long, unstructured document changes topics without visible section markers.

Consider:

```

Customers can return eligible products within 30 days.
Electronic products must be returned within 7 days.
Returned products must be inspected before approval.

We offer standard shipping across India.

```

Orders are generally dispatched within 24 hours.
Express delivery is available in selected cities.

The first group discusses returns; the second discusses shipping. A conceptual semantic-chunking process is:

```
Split text into sentences or small groups
      ↓
Generate embeddings for neighboring groups
      ↓
Compare their semantic similarity
      ↓
Identify notable topic transitions
      ↓
Create topic-coherent chunks
```

Illustrative similarity scores:

Adjacent groups	Hypothetical similarity
Return sentence 1 ↔ return sentence 2	0.92
Return sentence 2 ↔ return sentence 3	0.87
Return sentence 3 ↔ shipping sentence 1	0.31
Shipping sentence 1 ↔ shipping sentence 2	0.94

The drop at the returns-to-shipping transition suggests a possible boundary. Semantic chunking can require additional embedding calls and processing, and it does not guarantee that tables, cross-references or exceptions remain intact.

2.5 The chunk-size trade-off

Smaller chunks	Larger chunks
More specific searchable units	More surrounding context in each result
Risk losing surrounding conditions	Risk mixing unrelated topics
Can require retrieving several chunks for one answer	Consume more prompt tokens per retrieved chunk

Overlap can protect boundary context, but adds redundancy. **There is no universally correct chunk size:** test choices against the structure of the

documents and the questions customers actually ask.

Practical: Configure token-based chunking in Spring AI

```
TokenTextSplitter splitter = TokenTextSplitter.builder()
    .withChunkSize(400)
    .withMinChunkSizeChars(150)
    .withKeepSeparator(true)
    .build();

List<Document> chunks = splitter.apply(originalDocuments);
vectorStore.add(chunks);
```

Setting	Purpose
<code>withChunkSize(400)</code>	Sets a target chunk size of 400 tokens.
<code>withMinChunkSizeChars(150)</code>	Controls the minimum character length considered while seeking suitable split boundaries; it is not an overlap setting or a strict minimum for every result.
<code>withKeepSeparator(true)</code>	Retains separators such as newlines.

Demo caution: Delete the previous vectors or index new chunks into a separate experimental namespace before comparing results. Otherwise, old and new chunk versions may appear together.

3. Improving Retrieval: Top-K, Similarity Threshold and Metadata Filtering

Once documents have been chunked and stored, retrieval configuration determines which chunks are eligible to reach the LLM.

3.1 Top-K: How many results should we request?

```
SearchRequest.builder()
    .query(question)
```

```
.topK(4)
.build();
```

`topK(4)` asks the vector store for **at most four highest-ranked matches**. Imagine these hypothetical scores for a return-policy question:

Rank	Retrieved document	Illustrative similarity
1	Electronics Return Policy	0.94
2	General Return Policy	0.89
3	Damaged Product Policy	0.85
4	Exchange Policy	0.79
5	Warranty Policy	0.68
6	Shipping Policy	0.42
7	Payment Policy	0.31

Setting `topK(3)` requests the first three matches; `topK(5)` requests up to five.

Increasing K is not automatically better. A large value may add irrelevant content, raise token consumption and increase cost or latency. A very small value may exclude a policy exception needed to answer the question. For this small support chatbot, compare **K = 3, 5 and 8** and inspect the actual retrieved chunks.

3.2 Similarity threshold: Are the closest results relevant enough?

A customer might ask: **“Who won the FIFA World Cup?”** The company knowledge base has no football content, yet nearest-neighbor search can still return the closest policy documents. *Closest* does not necessarily mean *relevant*.

A similarity threshold adds a minimum acceptance rule:

```
SearchRequest.builder()
    .query(question)
    .topK(4)
    .similarityThreshold(0.75)
    .build();
```

This requests up to four documents and discards results below the configured threshold.

Do not treat similarity as a correctness probability. A similarity of 0.90 does not mean there is a 90% chance that the answer is correct. Score behavior varies with the embedding model, vector store, distance metric and collection. A threshold such as 0.75 must be evaluated against real test queries, not assumed to be universally suitable.

If no documents qualify, handle the empty result explicitly:

```
if (relevantChunks.isEmpty()) {  
    return "I couldn't find relevant information "  
        + "in our company documents.";  
}
```

A result that passes the threshold still may not contain the *complete* answer, so generation instructions must continue to discourage unsupported claims.

3.3 Metadata filtering: Search the right subset

A document can carry structured attributes alongside its content, such as category, policy type and filename. These attributes let the application narrow the search space before or as part of vector retrieval.

```
new Document(  
    ""  
    "Electronic products can be returned  
    within 7 days of delivery."  
    "",  
    Map.of(  
        "category", "electronics",  
        "policyType", "returns",  
        "source", "return-policy"  
    )  
);
```

Example collection:

Document text	Category	Policy type
Electronic products can be returned within 7 days.	electronics	returns
Clothing products can be returned within 30 days.	clothing	returns
Refunds are initiated after inspection.	general	refunds

Practical: Attach metadata while loading PDFs

The following example extracts filename-derived fields, attaches them to each page and then splits the pages into chunks:

```
@PostConstruct
public void loadKnowledgeBase() {
    List<Document> allChunks = new ArrayList<>();

    TokenTextSplitter splitter = TokenTextSplitter.builder()
        .withChunkSize(300)
        .build();

    for (Resource resource : policyFiles) {
        String fileName = resource.getFilename();

        PagePdfDocumentReader reader = new PagePdfDocumentReader(resource);
        List<Document> pages = reader.read();

        String category = "general";
        String policyType = "general";

        if (fileName.contains("electronics")) {
            category = "electronics";
        }
        if (fileName.contains("return")) {
            policyType = "returns";
        }
    }
}
```

```

List<Document> updatedPages = new ArrayList<>();
for (Document page : pages) {
    Document updatedPage = page.mutate()
        .metadata("source", fileName)
        .metadata("category", category)
        .metadata("policyType", policyType)
        .build();

    updatedPages.add(updatedPage);
}

List<Document> chunks = splitter.apply(updatedPages);
allChunks.addAll(chunks);
}

vectorStore.add(allChunks);
}

```

The text is embedded for semantic retrieval; the structured metadata is available for filtering. The filename rules above are simple demo rules and depend on consistent filenames.

Filter a search to electronics:

```

List<Document> relevantChunks = vectorStore.similaritySearch(
    SearchRequest.builder()
        .query(question)
        .topK(5)
        .filterExpression("category == 'electronic
s'")
        .build()
);

```

Spring AI provides metadata-filter expressions and its supported Pinecone integration translates them into vector-store filters.

Avoid filters that are too restrictive. The correct answer for an electronics question may depend on both an electronics-specific policy and a general policy. In that case:

```
.filterExpression("category in ['electronics', 'general']")
```

If a customer has already selected a category in the UI, use that structured input directly. An extra LLM call is not necessary just to infer the same category again.

3.4 Combine the three controls

```
List<Document> relevantChunks = vectorStore.similaritySearch(
    SearchRequest.builder()
        .query(question)
        .topK(5)
        .similarityThreshold(0.70)
        .filterExpression(
            "category in ['electronics', 'genera
l']"
        )
        .build()
);
```

This example assumes every indexed chunk has a `category` field and the application already knows the query concerns electronics.

Control	Question it answers
Top-K	How many matches can be returned?
Similarity threshold	How relevant must a match be to qualify?
Metadata filter	Which subset of documents may be searched?

4. Hybrid Search: Semantic Search + Keyword Search

Why semantic search alone can miss an exact match

A customer asks: **"Does the AX-204 wireless headphone support fast charging?"**

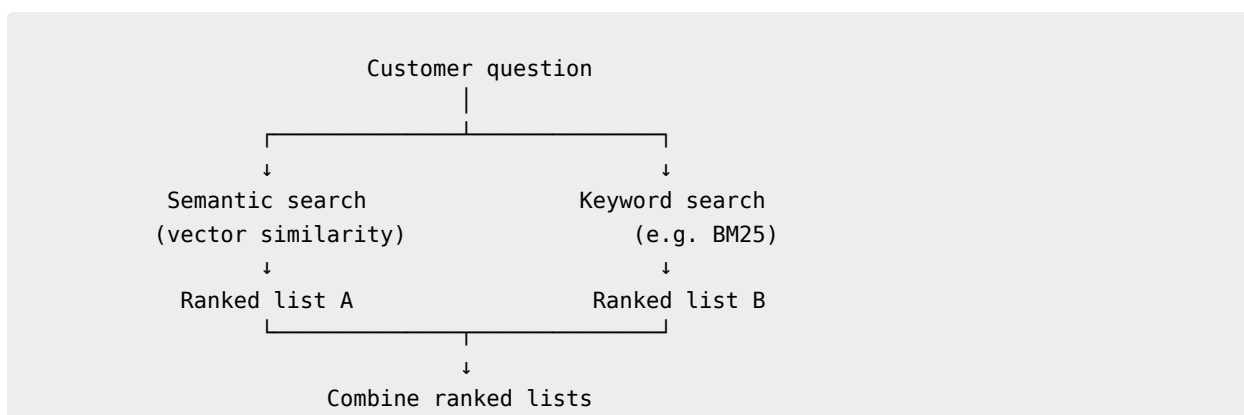
Document	Excerpt
A — AX-204 Product Manual	Model AX-204 supports standard USB-C charging. Fast charging is not supported.
B — BX-900 Product Manual	These wireless headphones support fast charging and provide five hours of playback after ten minutes of charging.
C — General Charging Guide	Wireless headphones supporting fast charging typically require a compatible adapter.

An embedding search may rank document B highly because it discusses *wireless headphones* and *fast charging*. However, the customer asked about the exact product code **AX-204**, not BX-900. Embeddings capture semantic relationships but are not guaranteed to preserve arbitrary identifiers reliably.

Two complementary retrieval methods

Semantic search	Keyword / lexical search
Matches concepts and meaning through embeddings.	Matches terms appearing in the query and documents.
Helps connect "money back" with "refund."	Helps find exact identifiers such as AX-204 .
Can rank a related but wrong product highly.	Can miss different expressions of the same meaning.

BM25 is a commonly used ranking method for lexical search. Hybrid search uses both kinds of retrieval so that semantic relationships and exact terms can contribute to the results.



↓
Final candidates

Reciprocal Rank Fusion (RRF)

Semantic search and BM25 produce scores on different scales; adding their raw scores directly is not necessarily meaningful. **RRF combines positions in ranked lists rather than their raw scores.**

For a document d , the standard idea is:

$$\frac{1}{\text{rank}_i(d)} = \sum_{i=1}^m \frac{1}{\text{rank}_i(d)}$$

- m is the number of result lists (two in our example: semantic and keyword search).
- $\text{rank}_i(d)$ is the document's position in the i -th list. A document not returned by a list contributes nothing for that list.
- k is a smoothing constant used by RRF; it is **not** the Top-K retrieval setting.
- The summation simply **adds the document's points from each list**. Higher positions contribute more points than lower ones.

A document appearing near the top of both lists tends to earn a stronger combined rank. This avoids directly comparing incompatible similarity and keyword scores.

Key idea: Semantic search helps find related meaning; keyword search helps preserve exact terms; RRF brings their rankings together.

5. Reranking: Reordering the Retrieved Candidates

5.1 The problem

For “Can I return my damaged headphones after 10 days?”, suppose initial retrieval returns:

Initial rank	Retrieved document
1	General Return Policy
2	Electronics Return Policy
3	Refund Processing Policy
4	Damaged Product Reporting Policy
5	Exchange Policy

The damaged-product document was retrieved, but it sits at position four. Taking only the top three would discard it. **Retrieval found a useful candidate, but its ranking was not ideal.**

5.2 Why add another ranking stage?

In embedding search, the query and stored documents are generally embedded independently. The document vectors can be created in advance, before any customer question arrives. That makes initial retrieval efficient for a large collection, but an embedding similarity score does not necessarily evaluate the full relationship between a particular question and a particular policy in detail.

A reranking model receives the **actual question and each candidate document** and assesses their relevance together. A common architecture is a *cross-encoder*, though not every reranker uses one.

Initial embedding retrieval	Cross-encoder reranking
Query and documents are represented independently.	The question and candidate document are processed jointly.
Document embeddings can be precomputed.	Relevance is assessed for the current question.
Efficient for searching a large collection.	More processing per candidate.
Produces an initial candidate list.	Reorders that candidate list.

5.3 Two-stage retrieval

```

Customer question
    ↓
Stage 1: Retrieve 10–20 candidate chunks
    ↓

```

Stage 2: Rerank those candidates against the question

↓

Select the top 3–5 reranked chunks

↓

Send selected context to the LLM

The candidate counts are illustrative, not universal settings.

For example, a cross-encoder could examine this pair:

Question:

Can I return my damaged headphones after 10 days?

Candidate document:

Damaged electronic products must be reported within
48 hours of delivery.

Illustrative reranker output:

Document	Initial rank	Hypothetical reranker score
General Return Policy	1	0.68
Electronics Return Policy	2	0.91
Refund Processing Policy	3	0.39
Damaged Product Reporting Policy	4	0.97
Exchange Policy	5	0.28

The damaged-product policy moves from fourth to first. Reranking does not create missing information; it changes the ordering of the candidates already retrieved.

Practical: Pinecone reranking with Java

The script demonstrates a hosted Pinecone reranker, `bge-reranker-v2-m3`, through its Inference API.

Step 1 — Retrieve more initial candidates:

```
List<Document> candidates = vectorStore.similaritySearch(  
    SearchRequest.builder()  
        .query(question)
```

```
        .topK(10)
        .build()
    );
```

Step 2 — Initialize the inference client:

```
Pinecone pc = new Pinecone.Builder(
    System.getenv("PINECONE_API_KEY")
).build();

Inference inference = pc.getInferenceClient();
```

Step 3 — Supply candidate IDs and text:

```
List<Map<String, Object>> documents = candidates.stream()
    .map(doc -> Map.<String, Object>.of(
        "id", doc.getId(),
        "text", doc.getText()
    ))
    .toList();
```

Step 4 — Request the four highest-ranked results:

```
RerankResult result = inference.rerank(
    "bge-reranker-v2-m3",
    question,
    documents,
    List.of("text"),
    4,
    true,
    Map.of()
);

System.out.println(result.getData());
```

Argument	Meaning
Model name	The hosted model used to rerank candidates.
<code>question</code>	The original customer query.
<code>documents</code>	The ten candidates from vector retrieval.
<code>List.of("text")</code>	The document field(s) used for ranking.
<code>4</code>	Return the top four candidates.
<code>true</code>	Include document content in the result.

These snippets show the demonstration flow; imports, dependency configuration and SDK-version details are not included in the source script. Compare the initial vector-search order with the reranked order when running the demo.

Reranking introduces another model operation and can add cost and latency. A very small knowledge base may not gain much from it.

6. Context Engineering: An Introduction

Retrieving the necessary documents is only part of the job. The LLM must also use them correctly.

Consider the question: **"Can I return my headphones after 10 days?"** The retriever returns:

Document	Retrieved information
General Return Policy	Products can be returned within 30 days of delivery.
Electronics Return Policy	Electronic products can be returned within 7 days of delivery.
Refund Policy	Approved refunds are processed within 5–7 business days.
Electronics Exchange Policy	Eligible electronic products can be exchanged within 7 days.

If the LLM responds, **"Yes, because the general return window is 30 days,"** it has ignored the category-specific policy even though both documents were retrieved.

This is a **generation failure rather than a retrieval failure**. Context engineering concerns the information we provide to the model and how we organize it so the model can interpret the relevant facts and constraints. It becomes important when context contains redundant, unrelated or competing information.

Scope of these notes: The provided script ends at this introduction to context engineering. It does not contain the subsequent implementation, so no additional techniques or code have been added as though they were part of the lecture.

Quick Reference

Technique	Main question	Typical failure it addresses
Better chunking	Where should documents be divided?	A rule and its exception end up in separate, incomplete results.
Top-K	How many candidate chunks should be retrieved?	Too few miss important rules; too many add noise.
Similarity threshold	Are even the closest results relevant?	An out-of-domain question still retrieves policy documents.
Metadata filtering	Which document subset is eligible?	Search ignores a known product category or policy type.
Hybrid search	How do we combine meaning and exact terms?	A similar product appears instead of the exact SKU.
RRF	How do we merge different ranked result lists?	Vector and lexical scores cannot be compared directly.
Reranking	Which retrieved candidates deserve the highest positions?	A useful policy is retrieved but ranked below less useful material.
Context engineering	How should retrieved material be supplied to the LLM?	The model ignores a specific policy in favor of a general one.

Practical principle: Evaluate retrieval and generation separately. First inspect what the database returned; then inspect how the model used it.